MULUNGUSHI UNIVERSITY

School of Engineering & Technology | Department of Computer Science

ICT261: Java Programming | Unit 4 Lab: Exception Handling and Error Management

Student Name: Pasche Simoonga
Student ID: 202509024
Lecturer: Mr. E Nyirenda
Date: November 2026

1. Activity A: Predict and Explain (5 Marks)

1. Analysis of Statements:

- `Integer.parseInt("ABC");` → **Exception:** `NumberFormatException`. **Classification:** Unchecked Exception (Subclass of `IllegalArgumentException`). **Strategy:** Wrap inside a `try-catch` block or validate string format using regular expressions prior to parsing.
- `String name = null; name.length();` → **Exception:** `NullPointerException`. **Classification:** Unchecked Exception (`RuntimeException`). **Strategy:** Perform a null safety check (`if (name != null)`) before invoking instance methods.
- `new java.io.FileReader("missing.txt");` → **Exception:** `FileNotFoundException`. **Classification:** Checked Exception (Subclass of `IOException`). **Strategy:** Handle via a `try-catch` block or declare `throws FileNotFoundException` in the method signature.

2. OutOfMemoryError Analysis:
`OutOfMemoryError` is a subclass of `Error`, indicating a severe JVM-level exhaustion (such as heap memory depletion) rather than an application-level exception. Catching it to continue execution is dangerous because the runtime state is unstable, risking cascading failures and data corruption.

2. Activity B & C: Decision Document & Architecture

Record Recovery Decision: Skip the invalid record and continue processing subsequent lines.

Justification: In batch data processing systems (such as student mark imports), stopping an entire file upload due to a single formatting error or typo halts administrative workflows unnecessarily. By wrapping row-level validation inside a localized `try-catch` block within the iteration loop, the application logs the error, informs the user securely, bypasses the malformed record, and preserves valid data entries.

Try-With-Resources Mechanism: The `try-with-resources` structure ensures that the `BufferedReader` and underlying file streams are automatically closed at the end of the execution block, preventing resource leaks even if unexpected exceptions occur.

3. Activity C: Test Output Verification Table

Test	Change / Input	Expected Behavior	Observed Result	Status
T1	`marks.txt` contains `ICT001,78`	Valid mark accepted	[ACCEPTED] ID: ICT001	Passed
T2	`ICT002,ABC`	Invalid numeric value reported safely	[REJECTED] Line 2 format error	Passed
T3	`ICT001,65` after `ICT001,78`	Duplicate student ID identified	[REJECTED] Duplicate ID	Passed

Test	Change / Input	Expected Behavior	Observed Result	Status
T4	`ICT004,120`	Out-of-range mark rejected	[REJECTED] Range 0-100 error	Passed
T5	Change filename to `absent.txt`	Friendly file-not-found; finally runs	Friendly error + finally msg	Passed

4. Activity D: Engineer's Review (5 Marks)

- **1. Checked Exception & Throws Location:** `IOException` (and `FileNotFoundException`) is checked. The `throws IOException` clause appears in `static void importMarks(String filename) throws IOException` because file I/O depends on external environment factors outside compiler certainty.
- **2. Throw vs. Throws:** `throw` explicitly triggers an exception instance (e.g., `throw new DuplicateStudentException(...);`), whereas `throws` declares method propagation in a signature (e.g., `throws IOException`).
- **3. Friendly Messages vs. Technical Logs:** Ordinary users need concise, actionable guidance (e.g., "File not found"), while technical logs via `java.util.logging.Logger` record detailed stack traces for developers without exposing system architecture.
- **4. Pitfalls:** Empty catch blocks silently swallow errors, masking critical bugs. Catching general `Exception` before specific `FileNotFoundException` causes compilation errors due to unreachable code blocks.